

FIAP

ENGENHARIA DE SOFTWARE

ALICE SANTOS BULHÕES	RM:554499
EDUARDO OLIVEIRA CARDOSO MADID	RM:556349
LUCAS HENZO IDE YUKI	RM:554865
NICOLAS HAUBRICHT HAINFELLNER	RM:556259

PROJETO TRACER

SÃO PAULO

2024

RESUMO

O projeto TRacer tem como objetivo popularizar a fórmula E por meio de um sistema desenvolvido para analisar dados de corridas. O sistema coleta informações como tempo de volta, velocidade média e horários precisos, utilizando uma conexão com uma máquina virtual (VM) para atualizar dados locais em um arquivo JSON. Mesmo sem a conexão com a VM, o sistema continua funcionando com dados pré-existentes, garantindo a continuidade das análises. O protótipo atual gera gráficos sobre a performance dos pilotos e pode ser expandido no futuro para incluir tempo real e rankings baseados no tempo de volta.

Sumário

1. Introdução	4
2. Finalidade do Projeto	4
3. Funcionalidades	5
3.1 Geração de Gráficos:.....	5
3.2 Submenu de Opções:	5
3.3 Gerenciamento de Conexão com a VM	6
4. Diagrama e gráficos gerados pelo programa	6
4.1 Gráfico de horário:	7
4.2 Gráfico de cada volta:	7
4.3 Gráfico de velocidade média:	8
5. Bibliotecas Necessárias	8
5.1 Matplotlib	8
5.2 Numpy	8
5.3 Scipy	9
5.5 JSON	9
5.6 OS e TIME	9
5.7 Datetime	9
6. Link para o GitHub	9
7. Vídeo no YouTube	9
9. Detalhes do Código.....	10
9.1 Estrutura de Arquivos.....	10
9.2 Tratamento de Erros	10

1.Introdução

O Projeto TRacer se baseia em um site e em um aplicativo nomeado TRacer, cujo objetivo é popularizar a Fórmula E. No aplicativo, haverá uma seção dedicada aos pilotos, onde será possível visualizar gráficos gerados a partir de um sistema desenvolvido em Python. Esse protótipo inicial do programa foi criado para analisar e visualizar dados de corridas, com foco na performance dos pilotos. O sistema coleta informações como tempo de volta, velocidade média e horários precisos em milissegundos, utilizando uma conexão com uma máquina virtual (VM) para atualizar um arquivo JSON local.

2. Finalidade do Projeto

O objetivo deste projeto é desenvolver um sistema capaz de analisar dados de corridas, como o tempo e a velocidade das voltas. O sistema utiliza uma requisição HTTP para uma VM que retorna um arquivo JSON e atualiza o arquivo dados.json interno. Dessa forma, mesmo que a VM esteja desconectada, ao invés de gerar um erro devido à falha de conexão, o sistema utiliza o arquivo JSON interno para gerar gráficos e mostrar as informações.

3. Funcionalidades

3.1 Geração de Gráficos:

- A partir dos dados das voltas, como horários ou tempos em milissegundos, três gráficos podem ser gerados:
 - Gráfico de volta em função do horário: Mostra o horário exato em que cada volta foi completada, incluindo milésimos. Uma linha azul é usada para suavizar a variação média do tempo de cada volta, facilitando a visualização.
 - Gráfico de tempo por volta: Exibe uma linha vermelha indicando o tempo exato de cada volta, enquanto uma linha azul pontilhada mostra a média dos tempos divididos pelo número de voltas.
 - Gráfico de velocidade média: Calcula a velocidade média das voltas, com base no comprimento da pista inserido pelo usuário. Isso permite flexibilidade para diferentes pistas.
-

3.2 Submenu de Opções:

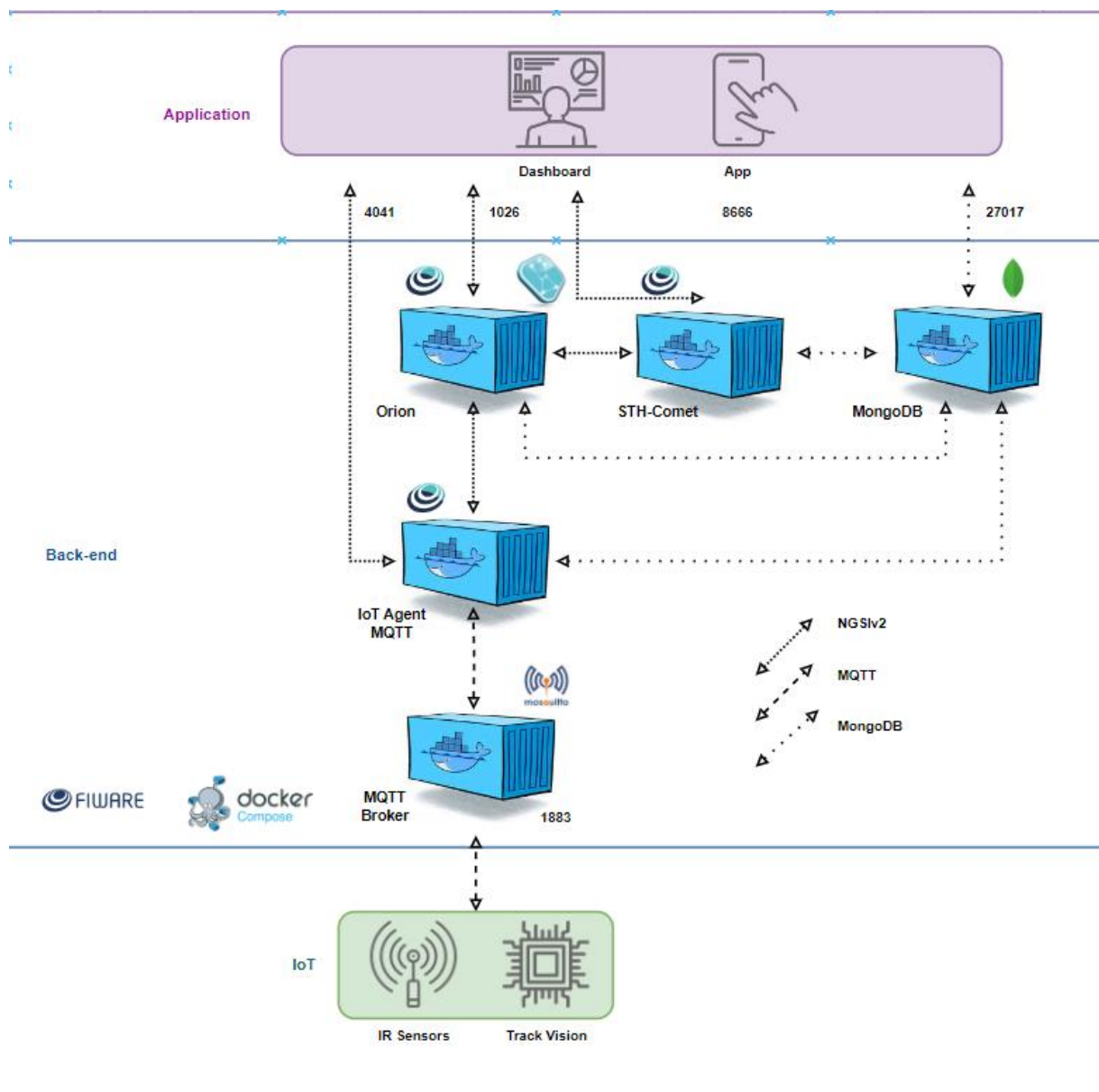
Dentro do terminal, uma das opções é apresentada como submenu, que consiste em outro menu dentro do principal, este coloca informações importantes impressas em texto.

- Volta mais rápida: Exibe a volta com o menor tempo de execução, junto com a respectiva velocidade média.
 - Volta mais lenta: Exibe a volta com o maior tempo de execução, juntamente com a menor velocidade média.
 - Informações de volta específica: O usuário pode selecionar uma volta específica, e o sistema exibe tanto a velocidade quanto a velocidade média dessa volta.
-

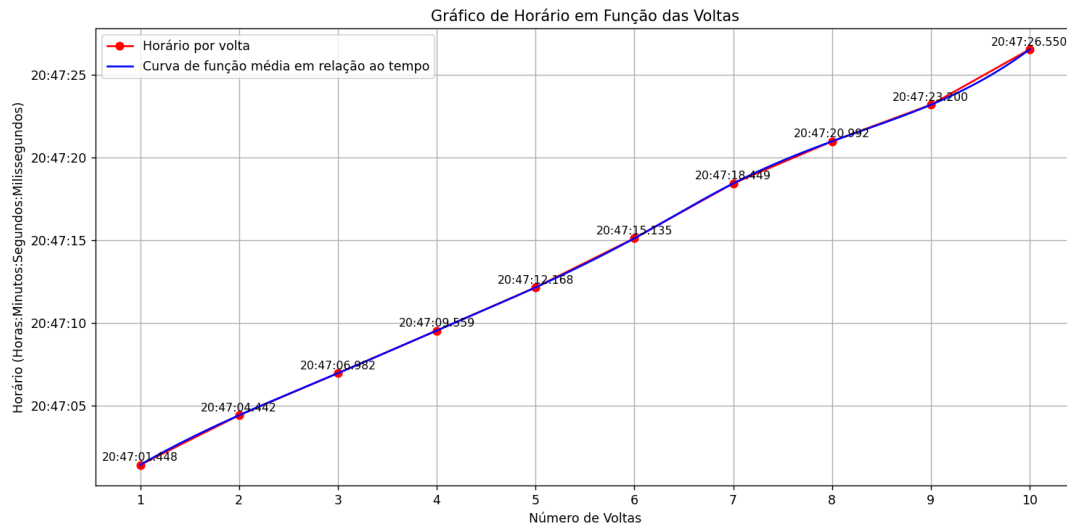
3.3 Gerenciamento de Conexão com a VM

- O sistema verifica a disponibilidade de conexão com a VM. Se houver conexão, os dados JSON são atualizados com as informações mais recentes.
- Se a conexão falhar, o sistema utiliza o JSON previamente salvo localmente para continuar fornecendo dados e gerando gráficos, evitando interrupções no serviço.

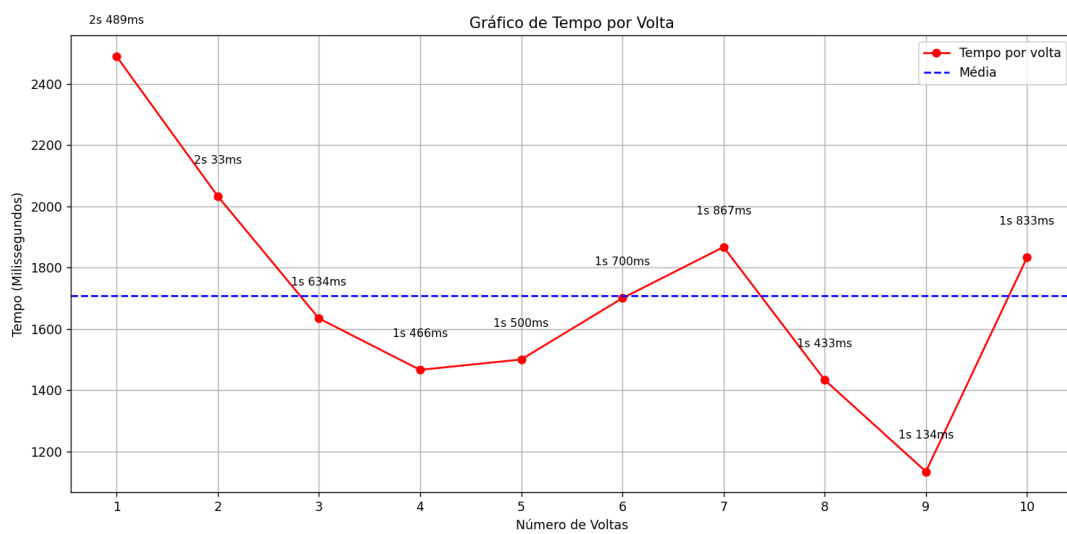
4. Diagrama e gráficos gerados pelo programa



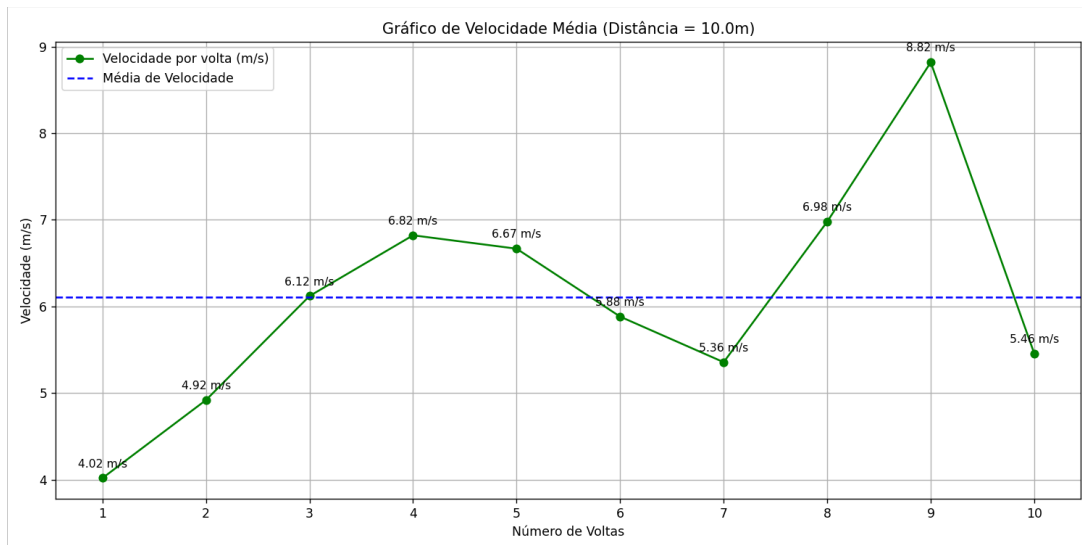
4.1 Gráfico de horário:



4.2 Gráfico de cada volta:



4.3 Gráfico de velocidade média:



5. Bibliotecas Necessárias

O sistema utiliza as seguintes bibliotecas para funcionar corretamente, todas devem ser instaladas diretamente pelo terminal:

5.1 Matplotlib

Para a criação e visualização de gráficos, incluindo funções de datas e formatação.

Instalação: `pip install matplotlib`

5.2 Numpy

Numpy: Para operações numéricas, como o uso de arrays e funções matemáticas.

Instalação: `pip install numpy`

5.3 Scipy

Scipy: Para interpolação de dados com a função `make_interp_spline`.

Instalação: `pip install scipy`

5.4 Requests

Requests: Para fazer requisições HTTP, como para obtenção de dados da VM.

Instalação: `pip install requests`

5.5 JSON

JSON: Biblioteca nativa do Python, utilizada para leitura e escrita de arquivos JSON.

5.6 OS e TIME

OS e Time: Bibliotecas nativas do Python para manipulação de arquivos e pausas temporárias no código.

5.7 Datetime

Datetime: Biblioteca nativa do Python para manipulação de datas e horários.

6. Link para o GitHub

[github](#)

7. Vídeo no YouTube

[youtube](#)

9. Detalhes do Código

9.1 Estrutura de Arquivos

- `main.py`: Arquivo principal que executa o sistema de análise.
 - `dados.json`: Arquivo JSON que contém os dados das voltas, utilizado caso a conexão com a VM falhe.
-

9.2 Tratamento de Erros

Cada funcionalidade realiza a validação de entradas antes de processar os dados para gerar estatísticas ou gráficos. Em caso de entradas inválidas, como uma volta inexistente, o sistema retorna mensagens amigáveis, orientando o usuário a corrigir o erro.
